A

OSOBA 1 / 3 · LIDER + KARTA A

Rozkład i wagon

`GET /api/events` · `GET /api/events/:id`

Ustawia wspólny grunt, a potem dowozi ekran, z którego klient wybiera kurs i wolne miejsce.

EFEKT	ZALEŻY OD	PRZEKAZUJE
Klik wolnego miejsca otwiera formularz B z poprawnym eventId i seatId.	Niczego — startuje pierwszy i publikuje kontrakt.	Kontrakt, fixtures, API i trasa do B.

Zadania

- ☐ **01 Zamknij kontrakt i szkielet repo**
  - packages/shared: typy, RESERVATION_TTL_MINUTES, isReservationActive, formatSeatLabel, fixtures.
  - CONTRACT.md: endpointy, błędy, trasy i ReservationView; 15-min review z O2 i O3.
- ☐ **02 Przygotuj dane demo i API A**
  - 3 kursy × 2 wagony (5 i 6) × 20 miejsc; numeracja 1–20, okno/korytarz z numeru.
  - GET /api/events i /:id; 404 dla nieznanego kursu; takenSeats z tego samego store'u co B.
- ☐ **03 Zbuduj listę kursów**
  - Relacja, data, odjazd, przyjazd; klik w kurs otwiera mapę wagonu.
  - Loading, empty i error; dane wyłącznie z API, bez hardkodowania w JSX.
- ☐ **04 Zbuduj mapę wagonu**
  - Przełącznik wagonów 5/6; siatka 5 rzędów × 4 miejsca; legenda okno/korytarz.
  - Zajęte wyszarzone i nieklikalne; wolne → /events/:eventId/reserve/:seatId.
  - Refetch po powrocie i na focus; ręczny przycisk odświeżania.
- ☐ **05 Spnij mapę z formularzem B**
  - Sprawdź, że aktywna rezerwacja B pojawia się w takenSeats po odświeżeniu.
  - Klik prowadzi do B z eventId, seatId i etykietą „Wagon 5, miejsce 12”.
- ☐ **06 Domknij jakość, ryzyka i demo**
  - Testy API A + test mapy; build bez błędów TS; README z uruchomieniem i TTL.
  - Wybierz 2–3 karty z listy zagrożeń; pokaż pełny przepływ na demo.
  - Lider: prowadź checkpointy integracji i pilnuj jednego źródła logiki.

GOTOWE, GDY...

- GET /api/events zwraca 3 kursy; /:id zwraca event + takenSeats, a nieznany id daje 404.
- Oba wagony mają po 20 miejsc; okno/korytarz są widoczne, zajęte miejsca zablokowane.
- Aktywna rezerwacja B blokuje miejsce; wygasła znika po odświeżeniu.
- Klik wolnego miejsca prowadzi do B z poprawnym eventId i seatId.
- Testy przechodzą, a README wystarcza do uruchomienia projektu.

Poza kartą A: jesteś właścicielem kontraktu i checkpointów integracji — nie implementujesz za innych.

Wspólny kontrakt w skrócie

`takenSeats`	`Odmowa zakupu`	`seatId`	`TTL`
string[]	**409**	**"5-12"**	**15 / 2 min**
kupione + ważne rezerwacje; bez wygasłych	RESERVATION_EXPIRED; miejsce wraca do puli	wagon-miejsce; eventId i seatId to tekst	RESERVATION_TTL_MINUTES ; testy ustawiają 2

Pełna wersja planu: Plan_Etapu_I_Pociag.pdf · kontrakt i API na s. 3–4.